

Análisis de Rendimientos de SO en Sistema Distribuido

Comparación de Linux, macOS y Windows ejecutando exactamente el mismo código, medida con OS Grid: un clúster de cómputo distribuido desarrollado por el equipo.

INTEGRANTES

Byron Jiménez Isaacs

Ronald Vargas González

PROFESOR

Carlos González Jara

Universidad Latina de Costa Rica

Contenido

- | | | | |
|----|---|----|---|
| 01 | Presentación del trabajo | 05 | Demostración en vivo |
| 02 | Sistema distribuido y análisis de rendimiento | 06 | Sistema interno: terminales y dashboard |
| 03 | Conexión de los equipos | 07 | Pruebas, métricas y resultados |
| 04 | OS Grid — Battle Royale | | |

— Objetivo: evaluar el rendimiento de tres sistemas operativos bajo una carga idéntica y reproducible.

Sistema distribuido y análisis de rendimiento

Una tarea pesada se divide en unidades de trabajo pequeñas — chunks — que se reparten entre varios nodos de ejecución. La estrategia de reparto determina el rendimiento global.

REPARTO ESTÁTICO

18 chunks divididos en cuotas fijas de 6. El tiempo total queda atado al nodo más lento; los rápidos quedan ociosos.

REPARTO DINÁMICO · WORK-STEALING

Cada nodo recibe trabajo apenas termina el anterior. El reparto se vuelve proporcional a la velocidad real de cada máquina.

ANALIZAR RENDIMIENTO = MEDIR

Tiempo por chunk

Mide la velocidad de la máquina, independiente del reparto.

Uso de CPU

Del proceso y del sistema, normalizado por núcleos lógicos.

Memoria residente (RSS)

Huella real del proceso worker durante el cómputo.

Varianza

Estabilidad de la medición entre ejecuciones.

Principio de validez: el mismo programa, los mismos parámetros, sin ramas por sistema operativo.

Conexión de los equipos

COORDINADOR · NODOFEDORA (LINUX)

FastAPI + Uvicorn · WebSocket `/ws` para workers y `/ws/dashboard` para el navegador · SQLite `os_grid.db`

Tailscale · VPN de malla · WebSocket sobre TCP

NodoFedora

Linux

`worker/agent.py`

NodoMac

macOS

`worker/agent.py`

NodoWindows

Windows

`worker/agent.py`

Tres máquinas físicas en redes distintas —sin virtualización, para no introducir una variable extra en la medición— unidas al mismo tailnet. El panel no permite arrancar hasta tener 3 / 3 equipos conectados.

STACK

COORDINADOR

Python 3.14 · FastAPI 0.141 · Uvicorn

WORKERS

websockets 17 · psutil 7.2

DASHBOARD

Angular 17 · TypeScript 5.4 · signals

PERSISTENCIA

SQLite

RED

Tailscale

Captura de la aplicación: pantalla de espera con el contador de equipos conectados.

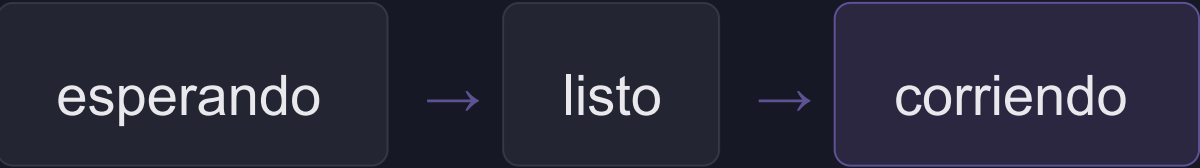
— 04 · SECCIÓN PRÁCTICA

OS Grid Battle Royale

Un clúster heterogéneo donde tres sistemas operativos compiten ejecutando el mismo código Python. Gana el que logra el menor tiempo promedio por chunk.

Cómo funciona el sistema

MÁQUINA DE ESTADOS DEL COORDINADOR



`MIN_WORKERS = 3` · al completarse todos los chunks vuelve solo a **listo** , lo que permite encadenar misiones sin reiniciar nada.

PROTOCOLO · JSON SOBRE WEBSOCKET

<code>REGISTER</code>	nombre, SO, núcleos lógicos, RAM total
<code>TASK_ASSIGN</code>	chunk: id, tipo, inicio, fin, tamaño, objetivo
<code>METRICS</code>	telemetría en vivo cada ~0.4 s
<code>RESULT</code>	tiempo y picos de CPU / RAM del chunk
<code>NO_MORE</code>	cola vacía; el worker espera la próxima misión

EL NÚCLEO: WORK-STEALING DINÁMICO

- 1 Al arrancar, el coordinador envía un chunk a cada worker.
- 2 Apenas un worker devuelve su `RESULT` , recibe el siguiente disponible.
- 3 El ciclo se repite hasta vaciar la cola de trabajo.

El nodo más rápido procesa naturalmente más chunks, sin que el coordinador conozca de antemano el hardware de cada máquina.

Por eso la comparación se hace por tiempo promedio por chunk y no por cantidad completada: la cantidad depende del reparto; el tiempo mide la máquina.

— 05

Demostración

PASO 1

Esperar 3 / 3 equipos

PASO 2

Elegir prueba e intensidad

PASO 3

Arrancar misión

PASO 4

Leer el marcador

Sistema interno

TERMINALES

```
coordinador · python -m coordinator.main  
→ chunk 04 · NodoMac  
→ chunk 05 · NodoWindows  
← RESULT chunk 04 · 0.122 s  
→ chunk 06 · NodoMac
```

Los tres workers corren en paralelo con el mismo comando; solo cambia el intérprete instalado. `detectar_os()` se usa únicamente para etiquetar métricas, nunca para alterar el camino de ejecución.

Capturas: terminal del coordinador y las tres terminales de workers en paralelo.

DASHBOARD · DOS PESTAÑAS

Panel principal

Progreso de la misión, configuración de prueba e intensidad, una tarjeta por nodo con sparkline de CPU, tabla de resultados con medallas y marcador acumulado.

Interacción SO — Battle Royale

Topología coordinador → workers con líneas que se iluminan al procesar, planificador rotulado por SO, cartel de ganador, leaderboard y consola de eventos en vivo.

TOLERANCIA A FALLOS

Contabilidad de chunks en vuelo

Reencolado automático

Vigilante cada 5 s · timeout 120 s

Máximo 3 intentos por chunk

Pruebas y medición

hash · SHA-256

Fuerza bruta sobre un rango. CPU pura, memoria constante: mide cómputo entero y planificador.

sort · Timsort

Ordena listas de floats en memoria. Estresa el asignador y la caché; mayor huella de RAM.

primos · criba

Eratóstenes segmentada sobre bytearray. Castiga la localidad de caché por patrones de salto.

TRES INTENSIDADES

PRUEBA	LIGERO	NORMAL	PESADO
hash	3 M · 6 chunks	12 M · 12	40 M · 18
sort	200 K · 6	1 M · 12	4 M · 18
primos	1 M · 6	5 M · 12	20 M · 18

En hash y primos el rango total se divide entre los chunks; en sort cada chunk ordena una lista completa, por lo que su carga total es $n \times \text{tam}$.

CÓMO SE MIDE

psutil en `worker/metrics.py`

CPU del proceso normalizada por núcleos, CPU del sistema y RSS.

Primera muestra a 0.15 s

Luego cada 0.4 s. Con el muestreo original de 0.5 s el nodo más rápido reportaba 0 % de CPU.

Cómputo en `ThreadPoolExecutor`

No busca paralelismo —el GIL lo impide— sino liberar el bucle de asyncio para seguir enviando telemetría.

14 misiones, 282 chunks

Datos medidos por OS Grid y persistidos en SQLite

SISTEMA	CHUNKS	T. PROMEDIO	MÍN · MÁX	CPU PROC	RAM	VICTORIAS
macOS	159	0.504 s	0.122 · 0.802	10.0 %	144.2 MB	14
Windows	70	1.386 s	0.228 · 3.781	8.4 %	100.7 MB	0
Linux	53	1.680 s	0.364 · 3.055	6.6 %	92.1 MB	0

2.7× y 3.3× más rápido

macOS frente a Windows y Linux por chunk; se llevó el 56 % del trabajo sin configuración manual.

Varianza de Windows

El rango más amplio (0.228–3.781 s), compatible con su planificador y procesos en segundo plano.

Limitación declarada

Los tres nodos tienen hardware distinto: el experimento no aísla SO de hardware. No hay un SO universalmente superior.